



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE: CPE 010 - Data Structures
and Algorithms

SECTION: CPE21S1

DATE PERFORMED: Sep 15, 2026

DATE SUBMITTED: Sep 15, 2026

NAME: Benj Federic De Leon

INSTRUCTOR: Engr. Jimlord M. Quejado

Seatwork 9.1 Implementing Trees

1. PROCEDURE OUTPUT

BinaryTree.h

```

C BinaryTree.h > getRootVal()
1  #ifndef BINARYTREE_H
2  #define BINARYTREE_H
3
4  template<typename T>
5  class BinaryTree{
6
7      private:
8          //data
9          T key;
10         // left
11         BinaryTree<T> *leftChild;
12         // right
13         BinaryTree<T> *rightChild;
14     public:
15
16         //Constructor:
17         BinaryTree( T rootObj);
18
19         //insert children
20         void insertLeft(T newNode);
21         void insertRight(T newNode);
22
23         //accessing the child nodes:
24         BinaryTree<T>* getLeftChild();
25         BinaryTree<T>* getRightChild();
26
27         // get and set node value
28         void setRootVal(T obj);
29         T getRootVal();
30
31         // checker if the node is a leaf or not
32         bool isLeaf();
33
34
35     };
36

```

```

37     //Constructor:
38     template<typename T>
39     BinaryTree<T>::BinaryTree(T rootObj){
40         //add a value to the key
41         key = rootObj;
42         //set your left and right child to nullptr
43         leftChild = nullptr;
44         rightChild = nullptr;
45     }
46 }
47
48 //Inserting left:
49 template<typename T>
50 void BinaryTree<T>::insertLeft(T newNode){
51
52     if(leftChild == nullptr){
53         //CREATE A NEW NODE
54         leftChild = new BinaryTree<T>(newNode);
55     }else{
56         BinaryTree<T>* newChild = new BinaryTree<T>(newNode);
57
58         //moves the existing left child below the new node
59         newChild->leftChild = leftChild;
60
61         //make the new node the left child;
62         leftChild = newChild;
63     }
64 }
65

```

```

65
66 //Inserting right:
67 template<typename T>
68 void BinaryTree<T>::insertRight(T newNode){
69
70     if (rightChild == nullptr){
71         rightChild = new BinaryTree<T>(newNode);
72     }
73     else{
74         BinaryTree<T>* newchild = new BinaryTree<T>(newNode);
75         newchild->rightChild = rightChild;
76         rightChild = newchild;
77     }
78 }
79
80 template <typename T>
81 BinaryTree<T>* BinaryTree<T>::getLeftChild(){
82     return leftChild;
83 }
84
85 template <typename T>
86 BinaryTree<T>* BinaryTree<T>::getRightChild(){
87     return rightChild;
88 }
89
90 // get and set node value
91 template <typename T>
92 void BinaryTree<T>::setRootVal(T obj){
93     key = obj;
94 }
95
96 template <typename T>
97 T BinaryTree<T>::getRootVal(){
98     return key;
99 }
100

```

```

100
101     //if leaf
102     template <typename T>
103     bool BinaryTree<T>::isLeaf(){
104         return leftChild == nullptr && rightChild == nullptr;
105     }
106
107 #endif

```

main.cpp

```

G main.cpp > ...
1  #include <iostream>
2  #include "BinaryTree.h"
3
4
5  /* E
6      A
7      / \
8      B  C
9      / \ / \
10     D E F NULLPTR
11  */
12  */
13
14  int main(){
15
16      //create the root node
17      BinaryTree<char>* root = new BinaryTree<char>('A');
18
19      //CREATE THE CHILDREN OF A
20      root->insertLeft('B');
21      root->insertRight('C');
22
23      //CREATE THE CHILDREN OF B
24      root->getLeftChild()->insertLeft('D');
25      root->getLeftChild()->insertRight('E');
26
27      //CREATE THE CHILDREN OF C
28      root->getRightChild()->insertLeft('F');
29
30      //PRINT TO MATCH PHOTO OUTPUT
31      std::cout << "Root: " << root->getRootVal() << std::endl;
32      std::cout << "Left Child of A: " << root->getLeftChild()->getRootVal() << std::endl;
33      std::cout << "Right Child of A: " << root->getRightChild()->getRootVal() << std::endl;
34      std::cout << "Right Child of C: " << root->getRightChild()->getLeftChild()->getRootVal() << std::endl;
35
36      return 0;
37

```

Output:

```

PS C:\Users\Benj Federic De Leon\Documents\
E21S1\HoA9p1\" ; if ($?) { g++ main.cpp -o
Root: A
Left Child of A: B
Right Child of A: C
Right Child of C: F
PS C:\Users\Benj Federic De Leon\Documents\

```

Concept	Description	Function
Binary Tree & Nodes	Hierarchical structure where a base Root node points to at most two child nodes, left and right.	Initializes a node with a value and sets both child pointers to nullptr.
Templates (template<typename T>)	C++ feature allowing classes to handle generic data types.	Enables the binary tree to store any data type without rewriting code.
Dynamic Memory (new)	Memory management allocation system on the heap.	Allocates node memory on the heap using pointers (BinaryTree<T>*).
insertLeft() / insertRight()	Method to attach lower nodes to a parent node.	Creates a child node; if a child exists, inserts the new node and shifts the existing child down.
getLeftChild() / getRightChild()	Subtree navigation methods.	Returns pointer references to child nodes.

getRootVal() / setRootVal()	Data encapsulation accessors.	Reads or modifies the value stored in the current node.
isLeaf()	Node status checker.	Returns true if both leftChild and rightChild are nullptr.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Was used as a guide for grammar, explaining concepts

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.